

Backend de datos dinámicos

Sably.co sobre Cloudflare D1 + Pages Functions

Connexis

7 de agosto de 2026

Índice

Backend de datos dinámicos sobre Cloudflare	1
Antes que nada: no hay login, y es a propósito	1
Por qué Cloudflare y no Supabase	2
Lo que hay montado	2
La API	3
Cumplimiento legal	4
Lo que falta	4
Operar el día a día	4
Migrar a Laravel después	5

Backend de datos dinámicos sobre Cloudflare

Estado a 2026-08-07. Base D1 sably-pulso creada y con esquema aplicado, API en Pages Functions escrita. Falta desplegar y cablear el frontend.

Antes que nada: no hay login, y es a propósito

Pediste saber “cómo loguearme”. La respuesta honesta es que **este backend no tiene pantalla de acceso**, y conviene explicar por qué antes de que te lo encuentres.

Un login implica usuarios, contraseñas, sesiones, recuperación de cuenta y segundo factor. Todo eso es superficie que hay que mantener y que se puede comprometer, y aquí sobra: el sistema solo lo usa una persona (tú) para una única acción (aprobar o rechazar). Para eso hay dos mecanismos sin contraseña:

Qué necesitas hacer	Cómo entras
Aprobar o rechazar un comentario	Enlace firmado que llega a tu correo
Ver la cola de moderación	El mismo enlace, o la consola de D1
Consultar leads	Consola de Cloudflare a D1 a Console
Ejecutar SQL puntual	wrangler d1 execute sably-pulso --remote --command "..."

El enlace firmado lleva un HMAC con caducidad. Quien no tenga la clave no puede fabricarlo, y si el correo se filtra el enlace caduca. Es el mismo patrón de “magic link” que usan Notion o Slack para invitar, sin la parte de crear cuenta.

AVISO: Un detalle que el diseño marcó como trampa: **el enlace del correo abre una página de confirmación, no ejecuta el cambio**. Los clientes de correo hacen prefetch de los enlaces, así que un GET que aprobara directamente aprobaría comentarios solo con que Gmail precargara el mensaje. Se aprueba con un POST desde esa página.

Cuando llegue Filament (Fase 2, Laravel 13) ahí sí habrá panel con login, y será el sitio natural para moderar. Este backend es puente: el contrato ya nace en `/api/v1` con los nombres y verbos que tendrá Laravel, y las tablas usan clave primaria ULID para mapear 1:1 a Eloquent con HasUids.

Por qué Cloudflare y no Supabase

Tú ya lo habías decidido, y los números lo respaldan para este caso concreto:

	Cloudflare	Supabase
Origen	El mismo que sably.co a sin CORS ni preflight	Dominio aparte a CORS y una petición extra
Infra	Pages, R2 y el token ya están en el proyecto	Un proveedor y una factura más
Latencia	Ejecuta en el borde, junto al visitante	Región fija, salto transatlántico desde LATAM
Coste a este volumen	0 USD en free tier	0 USD hasta 500 MB, luego 25 USD

La ventaja real no es el precio, es que **las Pages Functions comparten origen con el sitio**. Eso elimina CORS, elimina la comprobación de `Origin` como control de seguridad y elimina un dominio que mantener.

Lo que hay montado

Base de datos: sably-pulso

0a173a39-b9cb-4d50-a865-b8465af5bad6 · 10 tablas, 4 vistas, 13 índices.

Tabla	Para qué
subject	Lista blanca de lo votable. Sin fila aquí, no se puede votar ni comentar
consent	La autorización con el texto literal que se mostró
vote	Votos de artículo, uno por visitante
comment	Comentarios, nacen en pending
lead	Formulario de contacto
purchase	Compras verificadas de Hotmart
review_token	Token de un solo uso para dejar reseña
course_review	Reseñas de curso, atadas a una compra
moderation_log	Quién aprobó qué y cuándo

Las dos decisiones que sostienen todo

1. No se almacenan promedios. Ni una columna con el rating medio, ni triggers que la mantengan. El promedio sale de una vista SQL sobre la tabla de votos, cada vez.

Un contador almacenado hay que actualizarlo en cada INSERT, UPDATE y DELETE, y basta olvidar un caso –borrar una reseña ya aprobada, por ejemplo– para publicar un número que no corresponde a ninguna fila. Sin contador no hay nada que desincronizar. A esta escala calcularlo es gratis.

2. Lo que lee Google se hornea en el build. El JSON-LD y las estrellas visibles viajan en el HTML servido, nunca se inyectan por JavaScript. Así el marcado es auditable abriendo `dist/`, y no depende de que un fetch responda a tiempo cuando pasa Googlebot.

La garantía contra lo que ya pasó

Este sitio publicó valoraciones inventadas: un hash del slug generaba “4.7 de 5 · 87 valoraciones” y eso alimentaba datos estructurados. El esquema está diseñado para que no pueda repetirse:

```
purchase_id TEXT NOT NULL UNIQUE REFERENCES purchase(id)
```

Una reseña sin compra detrás **no es un caso que validar en código: es una fila que la base rechaza**. Y Article no recupera aggregateRating nunca – no es cuestión de umbral, es que Google no da fragmentos de reseña para ese tipo, que es exactamente lo que reportó Search Console.

Para que Course vuelva a mostrar estrellas hacen falta **5 reseñas aprobadas a mano, cada una atada por clave foránea a una compra real de Hotmart**.

La API

Todo bajo `https://sably.co/api/v1/`, mismo origen que el sitio.

Método	Ruta	Qué hace	Protección
POST	<code>/votos</code>	Vota 1–5 un artículo	Turnstile, límite por IP, 3 s mínimo de lectura
GET	<code>/pulso?subject=</code>	Devuelve votos y promedio	Pública
POST	<code>/comentarios</code>	Deja comentario (queda pendiente)	Turnstile, límite por IP, puntuación de spam
GET	<code>/comentarios?subject=</code>	Comentarios aprobados, sin correos	Pública
POST	<code>/leads</code>	Formulario de contacto	Turnstile, dedupe por correo y día
GET	<code>/snapshot</code>	Todo lo publicable, para el build	Bearer SNAPSHOT_TOKEN
POST	<code>/subjects</code>	Siembra el catálogo	Bearer SNAPSHOT_TOKEN

Cómo se protege sin cuentas de usuario

- **Turnstile** de Cloudflare, que no pide resolver puzzles. El script se carga solo al abrir el formulario: en el layout serían ~70 KB contra un presupuesto total de 50 KB de JS.
- **Límite por IP** contando sobre la propia tabla destino: 5 comentarios/hora, 10 leads/hora, 30 votos/hora.
- **Tiempo mínimo de lectura** como CHECK de la base, no como if del endpoint: un voto a los 400 ms no se puede insertar ni aunque el código falle.
- **La IP no se almacena**. Se guarda SHA-256(ip + sal + día), que permite exigir IPs distintas y limitar por origen sin conservar un dato personal, y que al cambiar cada día tampoco sirve para seguir a nadie.

Cumplimiento legal

La Ley 1581/2012 colombiana obliga a **poder probar** la autorización, no solo a pedirla, y el RGPD art. 7.1 dice lo mismo. Por eso consent guarda la versión de la política y **el texto literal que vio la persona**, no un booleano.

purge_after fija el plazo de conservación por fila: 90 días para el correo de quien comenta, 2 años para un lead comercial. Un cron los borra, así que el cumplimiento no depende de que alguien se acuerde.

Lo que falta

Paso	Estado
Base D1 creada y migrada	[hecho] hecho
Endpoints escritos	[hecho] hecho
Binding D1 en wrangler.toml	[hecho] hecho
Secretos en Cloudflare (TURNSTILE_SECRET, SNAPSHOT_TOKEN, IP_SALT)	[pendiente] te toca
Clave de sitio de Turnstile	[pendiente] te toca , se crea en el panel
Cablear ArticleRating y LeadModal a la API	[pendiente] pendiente
Formulario de comentarios	[pendiente] pendiente
Snapshot y horneado en el build	[pendiente] pendiente
Webhook de Hotmart y reseñas	[pendiente] pendiente de verificar el webhook

Lo que se decidió NO construir ahora

- **Login y panel de administración:** lo cubre Filament en la Fase 2.
- **Respuestas anidadas de más de un nivel:** complica la moderación sin aportar.
- **Notificaciones por comentario nuevo:** el correo de moderación ya avisa.
- **Búsqueda dentro de comentarios:** con este volumen no hace falta.

Operar el día a día

Ver la cola de moderación

```
wrangler d1 execute sably-pulso --remote \  
  --command "SELECT * FROM v_moderacion ORDER BY created_at DESC LIMIT 20"
```

Aprobar un comentario a mano

```
wrangler d1 execute sably-pulso --remote \  
  --command "UPDATE comment SET status='approved', moderated_at=unixepoch() WHERE id='<ULID>'"
```

Leads de la última semana

```
wrangler d1 execute sably-pulso --remote \  
  --command "SELECT name, email, course_interest, country FROM lead WHERE created_at > unixepoch()-604800"
```

Purgar lo que cumplió su plazo

```
wrangler d1 execute sably-pulso --remote \  
  --command "DELETE FROM comment WHERE purge_after < unixepoch(); DELETE FROM lead WHERE purge_after < unixepoch();"
```

También sirve la consola web: **Cloudflare a Workers & Pages a D1 a sably-pulso a Console**, que no requiere instalar nada.

Migrar a Laravel después

Está pensado para que cueste poco:

- Las tablas van **en singular** y con **PK de texto ULID**, que es lo que espera Eloquent con HasUids.
- Los timestamps son INTEGER unixepoch UTC a `to_timestamp()` en PostgreSQL.
- El contrato ya es `/api/v1` con los verbos y formas que tendrá Laravel.

Migrar se reduce a exportar el SQL, importarlo en PostgreSQL y cambiar `PUBLIC_API_BASE` en el frontend. Los componentes del sitio no se tocan.